Research Report

Portfolio task 1(a)

Unit code: COS40005

Unit Name: Computing Technology Project A

Submission date: Oct 9

Student Name: Nguyen Hoang Phuc Anh

Student ID: 105551545

Semester: September - November 2026

How should a CI/CD pipeline for detection rules be designed so that it is well structured, fast and reproducible, and what does the literature say about the gates, feedback speed and environments that make pipelines reliable?

Nguyen Hoang Phuc Anh
Department of Computer Science
Swinburne University of Technology
Hanoi, Vietnam
phucanh120106@gmail.com

***Abstract*—Detection rules, the queries that turn security telemetry into alerts, are increasingly managed as code and released through CI/CD pipeline. This paper reviews the literature on how such a pipeline should be built and run, organized around three review questions: Which quality gates are used, and which are most effective? How do pipeline speed and feedback time affect developer behavior and adaption of automated checks? How do containerized and reproducible environments improve the reliability of CI/CD results. Drawing on empirical studies of continuous integration, work on containers and reproducible builds, and emerging research on Sigma rule quality, we find that layered gated with explicit blocking-versus-advisory policy are better supported than any single gate, that a ten-minute feedback target is the most widely cited guideline, and slow or noisy builds erode trust, and that containers improve reliability only when dependencies are pinned and results are checked for determinism.**

Keywords—continuous integration, CI/CD, detection-as-code, Sigma rules, quality gates, build feedback time, reproducibility, containers.

I. INTRODUCTION

A detection rule is a standing query that turns security telemetry into alerts. Teams increasingly manage these rules as software, being written in a neutral format such as Sigma, stored in Git, reviewed through pull requests, validated automatically and deployed to the SIEM by a pipeline. The appeal is the same as in application development, namely traceability, repeatability and earlier detection of mistakes. The risk is also the same, a pipeline that is slow, flaky or built differently on each run will be bypassed or ignored, and a pipe line whose checks are shallow give false assurance.

This report supports CAP-07:Detection-as-Code and support the project's central research question: **How should a CI/CD pipeline for detection rules be designed so that it is well structured, fast and reproducible, and what does the literature say about the gates, feedback speed and environments that make pipelines reliable?**

The review follows the structure of the PRISMA 2020 statement.

II. RESEARCH ON THE BUSINESS DOMAIN

Detection-as-Code: managing cybersecurity detection rules like software, in version control, with automated validation in a CI/CD pipeline. A pipeline is only useful if analysts trust it, run it often and get the same answer every time. This report covers 3 sub-questions:

Q1: What quality gates do CI/CD pipelines in the literature use, and which are most effective?

Q2: How do pipeline speed and feedback time affect developer behaviour and adoption of automated checks?

Q3: How do containerised and reproducible environments improve the reliability of CI/CD results?

Strengths and similar solution: Existing CI services such as Travis CI [1] and Jenkins [2] already provide the automation Detection-as-Code needs. About 40% of 34,544 GitHub projects used CI, and 94% of surveyed developers would use it again [1]. CI adoption was also associated with more merged pull requests and no observed loss of quality [3].

Challenges and gap: A review of 69 studies found 20 challenges, led by domain constraints, lack of suitable tools and cost [2]. Almost all of this work targets application software rather than security detections, and no study ranks quality gates directly. This project addresses that gap by applying CI/CD design choices to a detection pipeline and measuring the result.

Scope and method: The sources were identified by a combination of keywords used in Google Scholar search, such as "continuous integration", "quality gate", "static analysis", "build duration", "feedback time", "Docker" and "reproducibility". Every candidate was then verified on its publisher/repository site. Inclusion criteria were that the study needed to be an empirical study or systematic review, published between 2015 and 2021, and clearly related to one of the three questions.

Two limits apply: There are not many studies that focus on security detection pipelines directly as evidence is drawn from general CI/CD research. A number of studies are observational, which means that the findings are associations and not actual cause.

III. RESEARCH ON END USERS

The end users are assumed to be the detection engineers and analysts who write, review and run the rules

Q1. What quality gates do CI/CD pipelines in the literature use, and which are most effective?

Waiting is acknowledged as a cost of CI, and the best proof is how developers don't like waiting time, but check themselves. There are few direct groundings that address the impact of build time on behaviour, and part of this section is inference.

How long builds take and why: The average CI build on Travis, found during our large GitHub study, was just under 500 seconds-failing builds took more time than passing builds. When it comes to waiting for builds, the authors of the papers claim to waste developer time [1]. Ghaleb identified that these factors relate to project size, team size, build configuration size and test density (number of tests to build), among others, and that developers could switch to other work while waiting [6]. The systematic review shows that one out of six research-based themes in CI is "reducing build and test time" and outlines strategies like making test dependencies explicit, choosing tests and prioritising tests [2].

What speeds feedback up: Jin and Servant compared 14 variants of 10 CI-improvement techniques on 100 real projects. Skipping builds gave large savings because build preparation takes a long total of time and test prioritisation gave developers early failure signals. But skipping steps also comes with trade-offs, it can miss failing tests. A technique strong on cost may be weak on feedback time and a technique low on cost may give out results poorly [7].

Effect on behaviour and adoption: Reliable and quick feedback appears to support adoption. Projects using CI pipeline released .54 per month, compared to .24 released for projects that do not use CI pipeline, they merged accepted pull requests 1.6 hours sooner . The same study found out the lack of people who use CI mainly are not familiar with it (47%) and missing automatic tests (44%) and CI acts as “watch dog” for detecting what developers tend to skip by mistake [1]. Among the CI bad practices listed by Zampetti the most common smell category comprises "ignored build failures" (29 smells) [5]. We can infer that if feedback is slow and unreliable, developers learn to ignore it.

IV. Research on the solution domain

The answer is having a controlled environment for running a CI/CD pipeline consisting of gates that are automated. The architecture, tools and design below are informed by the literature on gates C1 and containers C3.

Q2. What quality gates do CI/CD pipelines in the literature use, and which are most effective?

Automated testing and static analysis review gates are cited as being widely utilized and received support in the literature, while review-based gates are common. There's no study head to head, so most effective needs to be interpreted from the indirect evidence.

Automated tests and build status: In the 34,544 projects, they found that developers who use CI said it mainly stops them breaking builds (88%) and catches bugs earlier (80%). The biggest reason for not using it was the lack of automated tests (44%) [1]. A study of 246 Github projects and 182,056 pull requests found that CI adoption was involved with 20.5% more merged pull requests from core developers and 42.3% fewer rejected ones. While bug reports from external users did not change, reports from core developers rose 48%, which read as CI exposing defects [3]. Together, the test gate is suggested to improve both throughput and defect discovery without harming quality.

Static analysis and style checks: Zampetti studied how 20 open-source Java projects use static analysis tools inside CI pipelines, these checks give consistent, rule-based verdicts that fit well to the detection rules while keeping the cost low [4]. This corresponds to checking the rule syntax and schema validation.

Gate design matters as much as gate choice: Zampetti also built a catalogue of 79 CI bad smells from 13 practitioners, resulting in 533 relevant posts and 102 transcribed sentences. Which quality gates are set without developer input and build failure that are ignored are included [5]. A gate that people override or distrust give no protection, the systematic review adds that poor test quality (8 studies) and the lack of a test strategy (7 studies) were among the most reported challenges and that long-running and unstable test erode trust in results [2].

Q3. How do containerised and reproducible environments improve the reliability of CI/CD results?

The problem with the container model is that as long as the results are reproducible, it is achieved by freezing the environment in a versioned text file-this has to be done properly to gain such benefit. However, it is frequently a case of poorly written container definitions being a cause of failure.

Why containers help: Boettiger argues that Docker combines operating system-level virtualization, portable deployment, reuse and version management in one tool. Also containers share the host kernel, so they are much lighter than virtual machines. A Dockerfile is a short plain text script that records dependencies and can be kept in a version control system, taking up less space than text-based setup notes. Images carry tags and history that allow for automatic undo and rebuild, helping to detect code rot [8]. For a pipeline this means that every run, on any machine or CI runner, it starts from the same environment.

Where containers fall short: Cito analyzed over 70,000 Dockerfiles and found that the biggest quality issue was due to the lack of version pinning (28.6% of issues). In a sample of 540 Dockerfile, 34% could not be built and it takes longer for a successful build to finish, they recommend quality checks during the build process, such as warnings for unpinned versions [9]. Boettiger also notes that containers still rely on the host kernel, and that undo does not restore the previous state of a layer, so it is advisable to store image files for accurate snapshots [8].

How to build them well: Nüst give out ten rules for reproducible Dockerfiles and the most relevant here are: pin system, package and source versions; build on maintained base images; keep the Dockerfile and its inputs in version control; order instructions to use build caching; rebuild regularly, every one to two weeks and without cache to expose breaking changes early. And it is not possible to exactly recreate an environment from a Dockerfile years later [10]. Systematic reviews showed that production-like test environments are a reported challenge and only two studies used configuration management in their process [2]. This suggests that environment control is still not fully utilized in practice. Similarly, Zampetti listed misaligned local and

remote workspaces as one of the most important CI problem [5].

V. KOST ANALYSIS

The selected design needs the knowledge, skills and technology below

A. Knowledge

For knowledge I will have to learn about CI/CD concepts, quality port design, ATT&CK and Sigma rule structures, reproducibility principles and the plan to close the gap in my knowledge are: Research the sources in this report, review Sigma and ATT&CK documentation, and agree on port thresholds with the client.

B. Skills

Git workflow and peer review, writing pipeline configurations, writing Dockerfiles with pinned versions, and reading build logs are some of the skill I need to acquire and

practicing on a small pilot pipeline, adhering to ten Dockerfile rules may improve it

C. Technology

By using CI service, a containerized runtime environment, static testing and rule testing tools, and an isolated lab for attack simulation I can start my project's early stage I can start with a free CI package and publicly available base images, try tools early, and record pinned versions in the repository.

VI. ETHICAL CONSIDERATIONS

This process handles detection rules, test logs and simulated attacks, therefore, the ethical issues focus on privacy, authorized testing and secure handling of artifacts. The checklist lists the protocols that appear to be applied and how the project adheres to them, these need to be re-examined against the unit's ethics guidelines and client rules.

A. Personal information in logs

The Australian Privacy Principles require the use of synthetic or de-identified log data in tests to ensure real user data is not stored in the repository.

B. Attack simulation

The Cybercrime Act 2001 (Cth) requires written client authorisation, and simulations must be conducted in an isolated lab within a scope agreed upon with the client.

C. Pipeline security

Secure handling of secrets and container images involves keeping credentials out of the repository and using pinned, maintained base images.

D. Professional conduct

The ACS Code of Professional Conduct requires honest reporting of limitations and failures, including any missed detections.

VII. REFERENCES

[1] M. Hilton, T. Tunnell, K. Huang, D. Marinov and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2016. DOI: 10.1145/2970276.2970358.

[2] M. Shahin, M. A. Babar and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," IEEE Access, vol. 5, 2017. DOI: 10.1109/ACCESS.2017.2685629.

[3] B. Vasilescu, Y. Yu, H. Wang, P. Devanbu and V. Filkov, "Quality and productivity outcomes relating to continuous integration in GitHub," in Proc. 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE), 2015. DOI: 10.1145/2786805.2786850.

[4] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora and M. Di Penta, "How open source projects use static code analysis tools in continuous integration pipelines," in Proc. 14th Int. Conf. Mining Software Repositories (MSR), 2017.

[5] F. Zampetti, C. Vassallo, S. Panichella, G. Canfora, H. C. Gall and M. Di Penta, "An empirical characterization of bad practices in continuous integration," Empirical Software Engineering, 2020. DOI: 10.1007/s10664-019-09785-8.

[6] T. A. Ghaleb, D. A. da Costa and Y. Zou, "An empirical study of the long duration of continuous integration builds," Empirical Software Engineering, 2019. DOI: 10.1007/s10664-019-09695-9.

[7] X. Jin and F. Servant, "What helped, and what did not? An evaluation of the strategies to improve continuous integration," in Proc. 43rd Int. Conf. Software Engineering (ICSE), 2021. arXiv: 2102.06666.

[8] C. Boettiger, "An introduction to Docker for reproducible research," ACM SIGOPS Operating Systems Review, vol. 49, no. 1, 2015. arXiv: 1410.0846.

[9] J. Cito, G. Schermann, J. E. Wittern, P. Leitner, S. Zumberi and H. C. Gall, "An empirical analysis of the Docker container ecosystem on GitHub," in Proc. 14th Int. Conf. Mining Software Repositories (MSR), 2017.

[10] D. Nüst et al., "Ten simple rules for writing Dockerfiles for reproducible data science," PLOS Computational Biology, vol. 16, no. 11, e1008316, 2020. DOI: 10.1371/journal.pcbi.1008316.